KertKérdés projekt dokumentáció

ASP.NET Core MVC alapú kertészeti kérdés-válasz közösségi platform

Készítette:

Deák Laura

Hadházy Nikolett

Kerekes Judit

2026.04.28.

Tartalomjegyzék

Tartalomjegyzék 2
1. Bevezetés 5
2. Fejlesztési dokumentáció 7
2.1 Fejlesztői környezet leírása 7
Fejlesztői gépen használt szoftverek 7
Visual Studio 2022 7
GitHub verziókezelés 8
SQL Server Management Studio 8
Szerveroldali futtatási környezet 8
Szerveroldali környezet 9
Session kezelés 9
2.2 Választott technológia – ASP.NET Core MVC architektúra részletes bemutatása 11
Az MVC működési elve 12
Controller réteg szerepe a projektben 12
FiokController.cs 12
KerdesController.cs 13
ModeratorController.cs 13
ValaszController.cs 13
Model réteg szerepe 14
Felhasznalo.cs 14
Kerdes.cs 14
Valasz.cs 14
Cimke.cs 14
KerdesCimke.cs 15
Temakor.cs 15
View réteg szerepe 15
Views/Kerdes/ 15
Views/Fiok/ 16
Views/Moderator/ 16
Razor View technológia 16
Data réteg – adatbázis kapcsolat 17

AppDbContext.cs 17
Helper osztályok szerepe 17
JelszoHelper.cs 17
CimkeModeraloHelper.cs 18
Program.cs szerepe 18
Miért számít ez nagyvállalati technológiának? 18
2.3 Kiegészítő technológiák 20
Entity Framework Core 20
SHA256 jelszóhash 21
Szövegmoderáció 22
2.4 Az adatbázis felépítése 23
Felhasznalok 24
Kerdesek 24
Valaszok 25
Cimkek 25
KerdesCimkek 25
Temakorok 25
Szavazatok 25
Adatbázis diagram 26
2.5 Kiválasztott funkció részletes ismertetése – kérdés létrehozás folyamata 27
A folyamat lépései 28
1. Felhasználói űrlap kitöltése 28
2. Controller fogadja az adatokat 30
3. Moderációs ellenőrzés 31
4. Címkék feldolgozása 31
5. Adatbázis mentés 33
AppDbContext részletes működése és szerepe a projektben 33
Miért van szükség DbContext osztályra? 33
Mit jelent pontosan az Add() művelet? 34
DbSet-ek szerepe – az adattáblák leképezése 37
OnModelCreating szerepe 37
Kapcsolatok részletes magyarázata 38
Szavazatok konfigurációja 39
Egyedi indexek szerepe 40

Kérdés szavazat index 40
2.6 Tesztelési dokumentáció 41
Tesztelés 41
3. Felhasználói dokumentáció 47
Első indítás menete 49
Belépés menete 49
Végfelhasználói használat – publikált alkalmazásként 49
Publish eredménye 49
IIS használata 50
Hogyan működik IIS alatt? 50
3.3 Felületek bemutatása 51
főoldal 52
regisztráció 54
bejelentkezés 56
kérdés létrehozás 57
kérdés részletek 59
moderálás felület 61
4. Összefoglalás 64
Továbbfejlesztési lehetőségek 64

1. Bevezetés

A projektünk címe KertKérdés, amely egy kertészeti témájú webalkalmazás, és elsődleges célja egy olyan online felület létrehozása, ahol a felhasználók növénytermesztéssel kapcsolatos kérdéseket tehetnek fel, illetve válaszokat adhatnak egymás kérdéseire. Az alkalmazás alapgondolata abból indult ki, hogy a kertészkedéshez kapcsolódó gyakorlati problémák sokszor nagyon speciálisak, és az interneten található általános leírások nem minden esetben adnak közvetlenül használható választ. Egy közösségi alapú kérdés–válasz rendszer ezzel szemben lehetővé teszi, hogy valós tapasztalatok alapján szülessenek megoldások.

A rendszer célja nem egyszerűen egy fórum létrehozása volt, hanem egy olyan strukturált webes alkalmazás elkészítése, ahol a kérdések témakörök szerint rendezhetők, címkékkel elláthatók, kereshetők, és a tartalmak moderáció után jelennek meg. Ezáltal a felület áttekinthető marad, és hosszabb távon egy jól használható tudásbázissá válhat.

A témaválasztás mellett szólt az is, hogy a kertészkedés egyre népszerűbb tevékenység, különösen az otthoni növénytermesztés, balkonkertek és fenntartható életmód iránt érdeklődők körében. A felhasználók között lehetnek kezdők, akik alapvető növényápolási kérdésekre keresnek választ, de olyan haladóbb érdeklődők is, akik speciális problémák megoldásában szeretnének segítséget kérni.

A projekt megtervezésekor fontos szempont volt, hogy ne csak egyszerű statikus weboldalt készítsünk, hanem olyan dinamikus alkalmazást, amely adatbázisra épül, felhasználókezelést tartalmaz, jogosultságokat különböztet meg, és képes hosszabb távon bővíthető szerkezetben működni. Ennek érdekében a fejlesztés során modern szerveroldali webtechnológiát választottunk.

A projekt célközönsége elsősorban:

- hobbi kertészek,
- kezdő növénytartók,
- házikerti termelők,
- illetve minden olyan felhasználó, aki gyors, gyakorlatias segítséget szeretne kapni.

A témaválasztás mellett szólt az is, hogy a kertészkedés napjainkban egyre népszerűbb, különösen az otthoni önellátás és fenntarthatóság előtérbe kerülése miatt.

A fejlesztés során olyan rendszert terveztünk megvalósítani, amely:

- regisztrációt és belépést kezel,
- különböző jogosultsági szinteket támogat,
- kérdésfeltevést tesz lehetővé,
- címkézést használ a könnyebb kereshetőség érdekében,
- moderációs funkciókat tartalmaz,
- szűrhető és kereshető adatbázist épít fel.

A projekt egyik fontos célja volt az is, hogy valódi webalkalmazás-architektúrát alkalmazzunk, tehát külön kezeljük:

- az adatbázist,
- az üzleti logikát,
- a felhasználói felületet.

Ezért MVC szerkezetben készült el az alkalmazás.

2. Fejlesztési dokumentáció

A projekt fejlesztése során ASP.NET Core MVC keretrendszert használtunk .NET 8 alapokon. A fejlesztéshez szükséges környezet szerveroldalon és kliensoldalon is több komponensből állt.

2.1 Fejlesztői környezet leírása

Fejlesztői gépen használt szoftverek

- Visual Studio 2022
- SQL Server Management Studio (SSMS)
- GitHub verziókezelés
- böngésző teszteléshez (Chrome / Edge)

Visual Studio 2022

A teljes projekt fejlesztése ebben a környezetben történt.

A Visual Studio előnyei:

- beépített hibakereső,
- IntelliSense kódkiegészítés,
- ASP.NET projektkezelés,
- közvetlen SQL kapcsolat.

Ez különösen hasznos volt a Controller és Model rétegek fejlesztésénél.

GitHub verziókezelés

A projekt verziókövetése GitHub segítségével történt.

Ez biztosította:

- a fejlesztési lépések visszakövethetőségét,
- a változtatások dokumentálását,
- a közös munkavégzés biztonságát.

SQL Server Management Studio

Az adatbázis kezelésére SQL Server környezetet használtunk.

Itt történt:

- adatbázis létrehozása,
- táblák ellenőrzése,
- adatok tesztelése.

Szerveroldali futtatási környezet

A projekt .NET 8 futtatási környezetet használ.

A szerveroldali működéshez szükséges:

- ASP.NET Core Runtime
- SQL Server adatbázis

Az alkalmazás konfigurációját az **appsettings.json** állomány tartalmazza.

Ebben található:

- connection string,
- adatbázis elérési útvonal.

Szerveroldali környezet

A projekt SQL Server adatbázist használ, amelyhez connection string van megadva az appsettings.json fájlban:

- SQL Server 2022
- Entity Framework Core adatkapcsolat

A kapcsolat konfigurációja appsettings fájlban történik.

```
1   {
2     "ConnectionStrings": {
3       "DefaultConnection": "Server=JUDITOS\\MSSQLDEV2022;Database=KertKerdesDb;
4     },
5
6     "Logging": {
7       "LogLevel": {
8         "Default": "Information",
9         "Microsoft.AspNetCore": "Warning"
10      }
11    },
12
13    "AllowedHosts": "*"
14  }
15
```

Session kezelés

A projekt session alapú felhasználókezelést használ:

- 30 perces timeout
- HttpOnly cookie
- alapvető session biztonsági beállítások

Ez a Program.cs állományban került konfigurálásra.

```
using KertKerdes.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseSession();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Kerdes}/{action=Index}/{id?}");

app.Run();
```

2.2 Választott technológia – ASP.NET Core MVC architektúra részletes bemutatása

A projekt fejlesztése során **a Microsoft által biztosított ASP.NET Core MVC keretrendszert** választottuk, amely napjaink egyik legelterjedtebb szerveroldali webalkalmazás-fejlesztési technológiája. Az MVC rövidítés a **Model – View – Controller** kifejezésekből származik, amely egy olyan architekturális minta, amelynek célja az alkalmazás különböző felelősségi köreinek szétválasztása.

Ez a megközelítés különösen fontos nagyobb rendszerek fejlesztésénél, mert lehetővé teszi, hogy az alkalmazás logikai részei jól elkülönüljenek egymástól, így a rendszer:

- könnyebben karbantartható,
- egyszerűbben bővíthető,
- több fejlesztő számára is átláthatóbb,
- hibakeresés szempontjából is hatékonyabb.

Az ASP.NET Core MVC technológia széles körben használatos vállalati környezetben is, mivel stabilitása, biztonsági megoldásai, teljesítménye és skálázhatósága alkalmassá teszi nagy felhasználószámú rendszerek kiszolgálására is. Nagyvállalati alkalmazásoknál gyakori követelmény, hogy a backend logika, az adatkezelés és a felhasználói megjelenítés egymástól függetlenül fejleszthető legyen. Az MVC ezt pontosan biztosítja.

Az MVC működési elve

Az alkalmazás működésének alapja, hogy a felhasználó böngészőből érkező kérése először egy Controller osztályhoz érkezik meg. A controller feldolgozza a kérést, szükség esetén adatokat kér le az adatbázisból a modelleken keresztül, majd kiválasztja azt a nézetet (View), amely a felhasználónak megjelenítésre kerül.

A folyamat: Felhasználó → Controller → Model → View → Felhasználó

Ez a működési logika a projektben minden fő funkciónál megfigyelhető.

Controller réteg szerepe a projektben

A Controller réteg felelős az alkalmazás vezérléséért. Itt történik:

- a felhasználói kérések fogadása,
- a logikai döntések meghozatala,
- az adatbázis műveletek indítása,
- valamint a megfelelő nézet kiválasztása.

A projektben a Controllers mappában találhatók ezek az állományok:

FiokController.cs

Ez kezeli:

- regisztrációt,
- bejelentkezést,
- kijelentkezést,
- felhasználói session létrehozását.

Ez különösen fontos, mert itt történik a hitelesítési folyamat.

KerdesController.cs

Ez a projekt egyik legfontosabb vezérlője, mivel itt kezeljük:

- kérdések listázását,
- új kérdés létrehozását,
- kérdés részleteinek megjelenítését,
- címkézést,
- keresést.

ModeratorController.cs

Ez a moderációs logikát kezeli:

- jóváhagyás,
- elutasítás,
- tartalom ellenőrzése.

Ez külön jogosultsági szintet képvisel a rendszerben.

ValaszController.cs

A válaszok létrehozását és kezelését végzi.

A controller réteg előnye, hogy minden üzleti logika központilag szervezett formában jelenik meg.

Model réteg szerepe

A Model réteg az adatstruktúrákat reprezentálja.

A Models mappában található osztályok közvetlenül megfeleltethetők az adatbázis tábláinak.

Felhasznalo.cs

A felhasználók adatait tartalmazza:

- név
- email
- jelszó
- szerepkör

Kerdes.cs

A kérdések adatait tárolja:

- cím
- leírás
- dátum
- státusz
- témakör

Valasz.cs

A válaszokat reprezentálja.

Cimke.cs

A címkézési rendszerhez tartozik.

KerdesCimke.cs

Kapcsolótábla szerepet tölt be, mivel egy kérdéshez több címke tartozhat.

Temakor.cs

Tematikus csoportosítás.

A modellek segítségével az adatbázis logikai szerkezete objektumorientált formában kezelhető. Ez azt jelenti, hogy például egy kérdés nem egyszerű rekordként jelenik meg, hanem egy teljes objektumként, amelyhez kapcsolódó válaszok és címkék is hozzárendelhetők. Ez jelentősen megkönnyíti a programozást.

View réteg szerepe

A View réteg a felhasználó számára látható felületet biztosítja. A projektben a Views mappában találhatók a nézetek.

Views/Kerdes/

Ebben találhatók:

- Index.cshtml
- Letrehozas.cshtml
- Reszletek.cshtml

Ezek jelenítik meg a kérdésekkel kapcsolatos felületeket.

Views/Fiok/

Tartalmazza:

- Bejelentkezes.cshtml
- Regisztracio.cshtml

Views/Moderator/

Moderációs képernyők.

Razor View technológia

A projekt View rétege Razor motorral működik. A Razor lehetővé teszi, hogy HTML és C# kód együtt jelenjen meg ugyanabban az állományban.

Példa:

- HTML szerkezet
- C# ciklusok
- feltételek

Ez nagyon hatékony, mert a dinamikus adatok közvetlenül beépíthetők az oldalba.

Data réteg – adatbázis kapcsolat

A projektben külön Data mappa található. Ez gyakorlatilag a rendszer adatmotorja.

AppDbContext.cs

Ez a teljes adatbázis kapcsolat központja.

Feladata:

- táblák összekapcsolása,
- relációk definiálása,
- Entity Framework működtetése.

Itt történik például:

- egyedi indexek megadása,
- kapcsolatkezelés,
- navigációs tulajdonságok konfigurálása.

Helper osztályok szerepe

A Helpers mappában olyan segédosztályok találhatók, amelyek külön logikai feladatokat látnak el.

JelszoHelper.cs

Feladata az SHA256 alapú jelszóhash készítés.

Ez azért fontos, mert a jelszavak nem kerülhetnek nyers formában adatbázisba.

CimkeModeraloHelper.cs

Feladata a tiltott szavak kiszűrése, címkék normalizálása. Ez moderációs funkció.

Program.cs szerepe

A Program.cs a teljes alkalmazás indulási pontja.

Itt történik:

- dependency injection konfigurálás,
- adatbázis kapcsolat inicializálás,
- session kezelés bekapcsolása,
- route kezelés.

Ez tulajdonképpen az alkalmazás indulási központja.

Miért számít ez nagyvállalati technológiának?

Az ASP.NET Core MVC olyan technológia, amelyet banki rendszerek, vállalatirányítási rendszerek, ügyfélportálok, belső adminisztrációs rendszerek esetében is alkalmaznak.

Ennek oka:

- Stabil szerveroldali architektúra
- Magas biztonsági szint
- Jól skálázható felépítés
- Könnyű tesztelhetőség
- Szabályozott projektstruktúra

A projektünk kicsiben ugyanazt az architektúrát használja, mint amit ipari rendszerekben is alkalmaznak.

A projekt során tudatosan választottuk az MVC architektúrát, mert ez biztosította azt, hogy az alkalmazás üzleti logikája, adatkezelése és felhasználói megjelenítése egymástól elkülönítetten, jól strukturált módon valósuljon meg, ami hosszú távon megkönnyíti a rendszer továbbfejlesztését és karbantartását.

2.3 Kiegészítő technológiák

A projektben az alap technológiák mellett megjelent:

Entity Framework Core

A DbContext kezeli az adatkapcsolatokat és relációkat.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Kerdes>()
        .HasOne(k => k.Felhasznalo)
        .WithMany(f => f.Kerdesek)
        .HasForeignKey(k => k.FelhasznaloId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Kerdes>()
        .HasOne(k => k.Temakor)
        .WithMany()
        .HasForeignKey(k => k.TemakorId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Valasz>()
        .HasOne(v => v.Felhasznalo)
        .WithMany(f => f.Valaszok)
        .HasForeignKey(v => v.FelhasznaloId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Valasz>()
        .HasOne(v => v.Kerdes)
        .WithMany(k => k.Valaszok)
        .HasForeignKey(v => v.KerdesId)
        .OnDelete(DeleteBehavior.Cascade);

    modelBuilder.Entity<KerdesCimke>()
        .HasOne(kc => kc.Kerdes)
        .WithMany(k => k.KerdesCimkek)
        .HasForeignKey(kc => kc.KerdesId)
        .OnDelete(DeleteBehavior.Cascade);

    modelBuilder.Entity<KerdesCimke>()
        .HasOne(kc => kc.Cimke)
        .WithMany(c => c.KerdesCimkek)
        .HasForeignKey(kc => kc.CimkeId)
        .OnDelete(DeleteBehavior.Cascade);
```

SHA256 jelszóhash

A jelszavak nem nyers formában kerülnek tárolásra.

A JelszoHelper SHA256 hash-elést végez:

```
1   using System.Security.Cryptography;
2   using System.Text;
3
4   namespace KertKerdes.Helpers
5   {
6       public static class JelszoHelper
7       {
8           public static string HashJelszo(string jelszo)
9           {
10              using (SHA256 sha256 = SHA256.Create())
11              {
12                  var bytes = Encoding.UTF8.GetBytes(jelszo);
13                  var hash = sha256.ComputeHash(bytes);
14
15                  var builder = new StringBuilder();
16
17                  foreach (var b in hash)
18                  {
19                      builder.Append(b.ToString("x2"));
20                  }
21
22                  return builder.ToString();
23              }
24          }
25      }
26  }
27
```

Ez adatbiztonsági szempontból fontos.

Szövegmoderáció

Tiltott szavak szűrése külön helper osztályban történik.

```
1   using System.Collections.Generic;
2   using System.Linq;
3
4   namespace KertKerdes.Helpers
5   {
6       public static class CimkeModeraloHelper
7       {
8           private static readonly List<string> AlapTiltottSzavak = new List<string>
9           {
10              "hülye","idiota","barom","szar","szarházi","kurva","geci","picsa","fasz","faszfej",
11              "basz","baszd","bazd","bazmeg","kibasz","kicsesz","szop","szopat","nyomorék",
12              "retardált","segg","seggfej","seggnyaló","balfasz","anyáz","anyád","anyádat","apád",
13              "rohad","rohadék","dög","dögölj","pusztulj","szemét","mocskos","kretén","gyökér",
14              "hülyegyerek","idióta","baromarc","féreg","patkány","csicska","csíra","szánalmas",
15              "szutyok","hányadék","okádék","fos","fosadék","szarfészek","kurvára","kurvás",
16              "ribanc","lotyó","ringyó","cafka","kurafi","bunkó","tapló","paraszt","suttyó"
17          };
18
19          private static readonly List<string> Ragok = new List<string>
20          {
21              "", "k", "ok", "ek", "ak", "ban", "ben", "ra", "re", "hoz", "hez", "höz",
22              "val", "vel", "ért", "ig", "ként", "t", "at", "et", "ot", "öt", "ul", "ül",
23              "ni", "nál", "nél", "ról", "ről"
24          };
25
26          private static readonly List<string> TiltottSzavak = GeneralTiltottLista();
27
28          private static List<string> GeneralTiltottLista()
29          {
30              var lista = new HashSet<string>();
31
32              foreach (var szo in AlapTiltottSzavak)
33              {
34                  foreach (var rag in Ragok)
35                  {
36                      lista.Add(szo + rag);
37                  }
38
39                  lista.Add(szo.Replace("a", "á"));
40                  lista.Add(szo.Replace("e", "é"));
41                  lista.Add(szo.Replace("i", "í"));
42                  lista.Add(szo.Replace("o", "ó"));
43                  lista.Add(szo.Replace("u", "ú"));
44              }
45
46              return lista.ToList();
47          }
```

Ez különösen hasznos közösségi platform esetén.

2.4 Az adatbázis felépítése

A KertKérdés alkalmazás relációs adatbázisra épül, amelynek célja a felhasználók, kérdések, válaszok, címkék és jogosultsági kapcsolatok strukturált tárolása. Az adatbázis kialakításánál fontos szempont volt, hogy a rendszer hosszabb távon is bővíthető maradjon, ezért az egyes funkciók külön táblákba kerültek szervezésre.

Az adatbázis kezelését az Entity Framework Core végzi, amely objektum-relációs leképezést biztosít. Ez azt jelenti, hogy az adatbázis táblái a programban C# osztályokként jelennek meg, így a fejlesztés során közvetlenül objektumokkal lehet dolgozni SQL utasítások helyett.

A központi adatkapcsolatot a Data/AppDbContext.cs állomány valósítja meg. Ez az osztály örökli a DbContext alaposztályt, és itt kerülnek deklarálásra a teljes rendszer adattáblái.

A projekt adatbázisa több relációs táblából áll.

Fő táblák:

- Felhasznalok
- Kerdesek
- Valaszok
- Temakorok
- Cimkek
- KerdesCimkek
- Szavazatok

Felhasznalok

Ez a tábla tárolja a rendszer felhasználóit.

Tárolt adatok:

- felhasználónév,
- email cím,
- jelszó hash,
- jogosultsági szint.

A jelszó nem nyers formában kerül tárolásra, hanem SHA256 algoritmussal titkosított karakterláncként. Ez biztonsági szempontból fontos, mert így adatbázis-szivárgás esetén sem olvasható vissza közvetlenül a felhasználó jelszava.

Kerdesek

Ez a rendszer egyik központi táblája. Minden feltett kérdés itt kerül eltárolásra.

Tárolt mezők:

- kérdés címe,
- leírás,
- létrehozás dátuma,
- státusz,
- felhasználó azonosító,
- témakör azonosító.

A státusz mező lehetővé teszi a moderációs folyamat kezelését. Ennek segítségével egy kérdés csak ellenőrzés után jelenik meg publikus formában.

Valaszok

A kérdésekhez kapcsolódó válaszokat külön tábla tárolja.

Kapcsolata: egy kérdéshez több válasz tartozhat. Ez klasszikus 1:N kapcsolat.

Cimkek

A címkézés célja a gyors kereshetőség. Egy kérdés több címkét is kaphat, például paradicsom, levéltetű, öntözés.

KerdesCimkek

Ez egy kapcsolótábla. Feladata a kérdések és címkék közötti N:M kapcsolat kezelése. Ez azért szükséges, mert egy kérdéshez több címke tartozhat, illetve egy címke több kérdésnél is használható.

Temakorok

A témakörök magasabb szintű besorolást adnak.

Például:

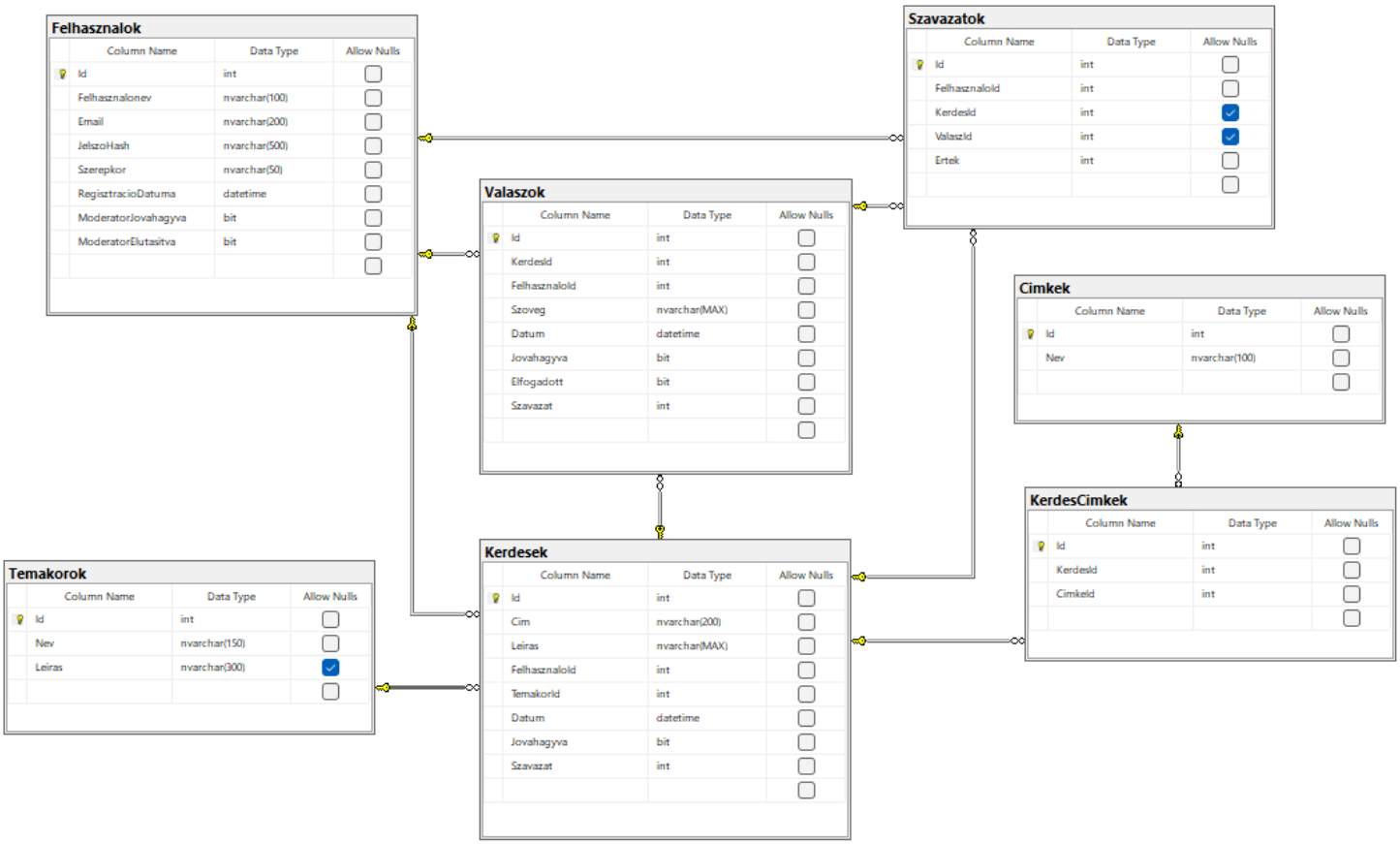
- növényvédelem
- talajkezelés
- öntözés

Ez segíti a navigációt.

Szavazatok

Külön táblában kezelve egyedi indexeléssel.

Adatbázis diagram



2.5 Kiválasztott funkció részletes ismertetése – kérdés létrehozás folyamata

A projekt egyik legösszetettebb része a kérdésfelvitel.

A rendszer ellenőrzi:

- be van-e jelentkezve a felhasználó
- kitöltötte-e a kötelező mezőket
- tartalmaz-e tiltott kifejezést

Ezután:

- létrehozza a kérdést
- automatikusan jóváhagyásra váró állapotba teszi
- címkéket normalizál
- duplikációt kiszűri
- kapcsolatot hoz létre a címkékkel

A folyamat különösen jól mutatja az MVC működését, mert:

- Controller vezérli
- Helper validál
- DbContext ment

Ezt a funkciót azért emeljük ki, mert ez a rész jól demonstrálja az alábbiakat:

- adatellenőrzés,
- helper logika,
- relációkezelés,
- többtáblás mentés működését.

Ez tipikusan olyan folyamat, amely vállalati rendszerekben is gyakori.

A folyamat lépései

1. Felhasználói űrlap kitöltése

A felhasználó a megfelelő nézeten keresztül adja meg:

- kérdés címét,
- részletes leírást,
- címkéket,
- témakört.

Ez a Views/Kerdes/Letrehozas.cshtml állományban történik.

```
<h2 class="mb-4">
    <i class="bi bi-plus-circle"></i>
    Új kérdés feltevése
</h2>

<form method="post">

    <div class="mb-4">

        <label class="form-label">
            <i class="bi bi-pencil"></i>
            Kérdés címe
        </label>

        <input asp-for="Cim"
               class="form-control form-control-lg" />

        <span asp-validation-for="Cim" class="text-danger"></span>

    </div>
```

```
<div class="mb-4">

    <label class="form-label">
        <i class="bi bi-card-text"></i>
        Kérdés leírása
    </label>

    <textarea asp-for="Leiras"
              class="form-control"
              rows="5"></textarea>

    <span asp-validation-for="Leiras" class="text-danger"></span>

</div>
```

```
<div class="mb-4">

    <label class="form-label">
        <i class="bi bi-tag"></i>
        Témakör
    </label>

    <select asp-for="TemakorId"
            class="form-select">

        @foreach (var temakor in ViewBag.Temakorok)
        {
            <option value="@temakor.Id">
                @temakor.Nev
            </option>
        }

    </select>

</div>
```

```
<div class="mb-4">

    <label class="form-label">
        <i class="bi bi-tags"></i>
        Címkék
    </label>

    <div class="form-text mb-2">
        A kérdéshez tartozó címkéket vesszővel elválasztva add meg.
        Például: paradicsom, levéltető, bio
    </div>

    <input type="text"
           name="ujCimkek"
           value="@(Context.Request.HasFormContentType? Context.Request.Form["ujCimkek"].ToString() : "")"
           class="form-control"
           placeholder="pl.: paradicsom, levéltető, bio" />

    @if (ViewData.ModelState.ContainsKey("ujCimkek"))
    {
        <span class="text-danger">
            @ViewData.ModelState["ujCimkek"]?.Errors.FirstOrDefault()?.ErrorMessage
        </span>
    }

</div>

<button type="submit"
        class="btn btn-success btn-lg">
    <i class="bi bi-send"></i>
    Beküldés
</button>
```

2. Controller fogadja az adatokat

A beküldött adat a **KerdesController.cs** megfelelő metódusába érkezik.

A controller feladata:

- ellenőrizni a kötelező mezőket,
- validálni az adatokat,
- elindítani a mentési folyamatot.

3. Moderációs ellenőrzés

A szöveg tartalmát a rendszer ellenőrzi.

Ehhez a **Helpers/CimkeModeraloHelper.cs** kerül használatra.

Feladata:

- tiltott szavak felismerése,
- címkék normalizálása.

```
List<string> egyediCimkek = new();

if (!string.IsNullOrWhiteSpace(ujCimkek))
{
    egyediCimkek = ujCimkek
        .Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(c => c.Trim().ToLower())
        .Distinct()
        .ToList();

    foreach (var cimkeNev in egyediCimkek)
    {
        if (CimkeModeraloHelper.TiltottCimke(cimkeNev))
        {
            ModelState.AddModelError("ujCimkek", "A címkék között tiltott kifejezés szerepel.");
            break;
        }
    }
}
```

4. Címkék feldolgozása

A címkéket külön bontja a rendszer.

Ha új címke érkezik → új rekord jön létre.
Ha már létezik → kapcsolat jön létre.

```
<input type="text"
       name="ujCimkek"
       value="@(Context.Request.HasFormContentType? Context.Request.Form["ujCimkek"].ToString() : "")"
       class="form-control"
       placeholder="pl.: paradicsom, levéltetű, bio" />

@if (ViewData.ModelState.ContainsKey("ujCimkek"))
{
    <span class="text-danger">
        @ViewData.ModelState["ujCimkek"]?.Errors.FirstOrDefault()?.ErrorMessage
    </span>
}
```

```
List<string> egyediCimkek = new();

if (!string.IsNullOrWhiteSpace(ujCimkek))
{
    egyediCimkek = ujCimkek
        .Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(c => c.Trim().ToLower())
        .Distinct()
        .ToList();

    foreach (var cimkeNev in egyediCimkek)
    {
        if (CimkeModeraloHelper.TiltottCimke(cimkeNev))
        {
            ModelState.AddModelError("ujCimkek", "A címkék között tiltott kifejezés szerepel.");
            break;
        }
    }
}

if (!ModelState.IsValid)
{
    ViewBag.Temakorok = _context.Temakorok.ToList();
    return View(kerdes);
}
```

A kérdés létrehozásakor a felhasználó vesszővel elválasztva adja meg a címkéket.

Ezeket először feldaraboljuk, kisbetűssé alakítjuk, majd kiszűrjük a duplikációkat.

```
foreach (var cimkeNev in egyediCimkek)
{
    var letezoCimke = _context.Cimkek.FirstOrDefault(c => c.Nev.ToLower() == cimkeNev);

    if (letezoCimke == null)
    {
        letezoCimke = new Cimke
        {
            Nev = cimkeNev
        };

        _context.Cimkek.Add(letezoCimke);
    }

    _context.KerdesCimkek.Add(new KerdesCimke
    {
        Kerdes = kerdes,
        Cimke = letezoCimke
    });
}

_context.SaveChanges();
```

Ezután minden címkén végigiterálunk. Először megnézzük, hogy létezik-e már ilyen címke a Cimkek táblában. Ha nem létezik, akkor új rekordot hozunk létre. Ez azért fontos, mert ugyanazt a címkét nem akarjuk többször eltárolni az adatbázisban. Ha a címke már létezik, akkor azt használjuk újra.

Ezután a KerdesCimkek kapcsolótáblába beszúrunk egy új rekordot, amely összekapcsolja az aktuális kérdést a címkével. Így jön létre a kapcsolat a kérdés és a címkék között. Ennek előnye, hogy rugalmas marad az adatmodell, mert bármennyi címke rendelhető egy kérdéshez, és ugyanaz a címke több kérdésnél is újra felhasználható.

5. Adatbázis mentés

Az adat végül AppDbContext segítségével kerül SQL Server adatbázisba.

AppDbContext részletes működése és szerepe a projektben

A projekt adatbázis-kezelésének központi eleme az **AppDbContext** osztály, amely a **Data/AppDbContext.cs** állományban található. Ez az osztály örökli a Microsoft Entity Framework Core által biztosított **DbContext** alaposztályt, amely lehetővé teszi, hogy az alkalmazás objektumorientált módon kommunikáljon az SQL Server adatbázissal.

Az AppDbContext feladata, hogy összekapcsolja a programban definiált modelleket az adatbázis megfelelő tábláival, valamint meghatározza az egyes táblák közötti kapcsolatok működését.

Miért van szükség DbContext osztályra?

Egy modern webalkalmazásban az adatbázis és a programkód között valamilyen közvetítő réteg szükséges. Ha ez nem lenne jelen, akkor minden adatbázis műveletet kézzel megírt SQL utasításokkal kellene megoldani.

Az Entity Framework Core ezt leegyszerűsíti: a programban objektumokkal dolgozunk, de a háttérben SQL lekérdezések jönnek létre.

Ez azt jelenti, hogy amikor például egy új kérdést mentünk, akkor a fejlesztő nem közvetlenül INSERT SQL utasítást ír, hanem a DbContext automatikusan generálja azt.

```
_context.Kerdesek.Add(kerdes);
_context.SaveChanges();
```

Ez jelentősen:

- csökkenti a hibalehetőséget,
- gyorsítja a fejlesztést,
- könnyebbé teszi a karbantartást.

Mit jelent pontosan az Add() művelet?

Amikor ehhez a sorhoz érünk:

```
_context.Kerdesek.Add(kerdes);
```

akkor valójában még **nem történik azonnali adatbázisba írás**.

Ebben a pillanatban csak az történik, hogy az Entity Framework:

- nyilvántartásba veszi az objektumot,
- "Added" állapotba teszi,
- eltárolja a memóriában, hogy ez egy új rekord lesz.

A Kerdes objektum ilyenkor már teljes C# objektum. Tartalmazza többek között a kérdés címét, leírását, felhasználó azonosítóját, témakört, státuszt.

```
public IActionResult Letrehozas(Kerdes kerdes, string ujCimkek)
{
    ModelState.Remove("ujCimkek");

    var felhasznaloId = HttpContext.Session.GetInt32("FelhasznaloId");

    if (felhasznaloId == null)
    {
        TempData["BelepesSzukseges"] = true;
        return RedirectToAction("Bejelentkezes", "Fiok");
    }

    if (CimkeModeraloHelper.TiltottCimke(kerdes.Cim))
    {
        ModelState.AddModelError("Cim", "A kérdés címe tiltott kifejezést tartalmaz.");
    }

    if (CimkeModeraloHelper.TiltottCimke(kerdes.Leiras))
    {
        ModelState.AddModelError("Leiras", "A kérdés leírása tiltott kifejezést tartalmaz.");
    }

    List<string> egyediCimkek = new();

    if (!string.IsNullOrWhiteSpace(ujCimkek))
    {
        egyediCimkek = ujCimkek
            .Split(',', StringSplitOptions.RemoveEmptyEntries)
            .Select(c => c.Trim().ToLower())
            .Distinct()
            .ToList();

        foreach (var cimkeNev in egyediCimkek)
        {
            if (CimkeModeraloHelper.TiltottCimke(cimkeNev))
            {
                ModelState.AddModelError("ujCimkek", "A címkék között tiltott kifejezés szerepel.");
                break;
            }
        }
    }
```

Tehát a programban először objektum jön létre, és csak utána kerül át adatbázisba.

Fontos szempont, mert az ORM (Object Relational Mapping) lényege, hogy nem rekordokkal dolgozik, hanem objektumokkal. Ez modern vállalati rendszerekben alapelv.

Mit csinál pontosan a SaveChanges()?

```
_context.SaveChanges();
```

Ez a sor indítja el a tényleges adatbázis műveletet.

Ekkor az Entity Framework:

1. Megvizsgálja, milyen objektumok változtak. Például: új rekord, módosított rekord, törölt rekord.

2. SQL utasítást generál

   Például kérdés létrehozáskor automatikusan ilyen SQL jön létre háttérben: INSERT INTO Kerdesek (...) VALUES (...)

3. Leküldi SQL Server felé

4. Visszaolvassa az új ID-t. Ha van automatikus elsődleges kulcs, azt visszaírja az objektumba.

A DbContextOptions tartalmazza:

- milyen adatbázist használ a rendszer,
- milyen connection string tartozik hozzá,
- milyen provider működik mögötte.

A projektben ez SQL Server kapcsolatot jelent. A kapcsolat tényleges beállítása a **Program.cs** állományban történik, ahol az alkalmazás indulásakor regisztrálásra kerül az AppDbContext.

Ez azért fontos, mert így dependency injection segítségével bármely controller megkaphatja az adatbázis kapcsolatot.

DbSet-ek szerepe – az adattáblák leképezése

Az AppDbContext-ben minden DbSet egy adatbázis táblának felel meg.

```
13 references
public DbSet<Kerdes> Kerdesek { get; set; }

14 references
public DbSet<Valasz> Valaszok { get; set; }

7 references
public DbSet<Felhasznalo> Felhasznalok { get; set; }

3 references
public DbSet<Temakor> Temakorok { get; set; }

2 references
public DbSet<Cimke> Cimkek { get; set; }

4 references
public DbSet<KerdesCimke> KerdesCimkek { get; set; }

12 references
public DbSet<Szavazat> Szavazatok { get; set; }
```

OnModelCreating szerepe

Az **OnModelCreating()** metódusban történik az adatbázis kapcsolatok részletes konfigurálása.

Ez az egyik legfontosabb része az egész projektnek.

Itt kerül meghatározásra:

- melyik tábla melyikhez kapcsolódik,
- milyen idegen kulcsok vannak,
- milyen törlési szabályok érvényesek.

Kapcsolatok részletes magyarázata

Kerdes → Felhasznalo kapcsolat

Ez azt jelenti: egy kérdés egy felhasználóhoz tartozik, viszont egy felhasználó több kérdést is létrehozhat.

Ez klasszikus 1:N kapcsolat.

```
0 references
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Kerdes>()
        .HasOne(k => k.Felhasznalo)
        .WithMany(f => f.Kerdesek)
        .HasForeignKey(k => k.FelhasznaloId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Kerdes>()
        .HasOne(k => k.Temakor)
        .WithMany()
        .HasForeignKey(k => k.TemakorId)
        .OnDelete(DeleteBehavior.Restrict);
```

Foreign key

Ez mondja meg, hogy a Kerdes táblában a FelhasznaloId mező kapcsolódik a Felhasznalok táblához.

```
.HasForeignKey(k => k.FelhasznaloId)
```

Restrict törlés

Ez azt jelenti: ha van kérdés a felhasználóhoz kapcsolva, a felhasználó nem törölhető automatikusan. Ez adatbiztonsági okból fontos.

```
 .OnDelete(DeleteBehavior.Restrict);
```

Kerdes → Temakor kapcsolat

A kérdés egy témakörhöz tartozik. Egy témakör viszont több kérdésnél is használható.

Valasz → Kerdes kapcsolat

```
modelBuilder.Entity<Valasz>()
    .HasOne(v => v.Kerdes)
    .WithMany(k => k.Valaszok)
    .HasForeignKey(v => v.KerdesId)
    .OnDelete(DeleteBehavior.Cascade);
```

Ez nagyon fontos.

```
.OnDelete(DeleteBehavior.Cascade);
```

Cascade jelentése: ha törlődik a kérdés, automatikusan törlődnek a hozzá tartozó válaszok is. Ez megakadályozza az árva rekordokat.

KerdesCimke kapcsolat

Ez a N:M kapcsolat megvalósítása. Miért kell külön tábla?

Ennek oka:

- egy kérdés több címkét kap,
- egy címke több kérdéshez tartozhat.

Szavazatok konfigurációja

Ez a projekt egyik technikailag legérdekesebb része. A Szavazat tábla kétféle objektumra is vonatkozhat:

- kérdésre,
- válaszra.

```
modelBuilder.Entity<Szavazat>()
    .HasIndex(s => new { s.FelhasznaloId, s.KerdesId })
    .HasFilter("[KerdesId] IS NOT NULL")
    .IsUnique();

modelBuilder.Entity<Szavazat>()
    .HasIndex(s => new { s.FelhasznaloId, s.ValaszId })
    .HasFilter("[ValaszId] IS NOT NULL")
    .IsUnique();
```

Ezért két külön idegen kulcs szerepel:

- KerdesId
- ValaszId

Itt is restrict-et használunk, mert nem akarjuk, hogy szavazatok véletlenül automatikusan törlődjenek.

Egyedi indexek szerepe

Ez biztosítja, hogy ugyanaz a felhasználó egy kérdésre csak egyszer szavazhasson.

Kérdés szavazat index

```
.HasIndex(s => new { s.FelhasznaloId, s.KerdesId })
```

Szűrő feltétel:

```
.HasFilter("[KerdesId] IS NOT NULL")
```

Ez azért kell, mert csak akkor alkalmazódjon, ha kérdéshez tartozik szavazat.

Egyediség

```
.IsUnique();
```

Ez garantálja, hogy nincs dupla szavazás.

Ez már tipikusan olyan adatbázis-tervezési szint, amit nagyobb rendszerekben is alkalmaznak. Itt már nemcsak adattárolás történik, hanem, hanem az üzleti szabályok adatbázis szinten is érvényesülnek.

Összegezve: az **AppDbContext osztály** a teljes alkalmazás adatkezelési központja, amely nemcsak a táblák leképezését végzi, hanem részletesen szabályozza az adatok közötti relációkat, a törlési viselkedést, valamint az üzleti szabályok adatbázis szintű érvényesítését is.

2.6 Tesztelési dokumentáció

A projekt során tesztelést végeztünk. A cél az volt, hogy minden fő funkció hibamentesen működjön.

Tesztelés

Teszt kód	Leírás	Lépések	Elvárt eredmény	Eredmény	Sikeres?
Regisztráció					
R001	Regisztráció helyes adatokkal, általános szerepkörű felhasználónak	Megadom nevet, e-mailt, jelszót, megerősítem, szerepkört választok	Új általános szerepkörű felhasználó létrejön, bejelentkezik	Létrejött	Igen
R002	Regisztráció helyes adatokkal moderátor szerepkörű felhasználónak	Megadom nevet, e-mailt, jelszót, megerősítem, szerepkört választok	Új moderátor szerepkörű felhasználó létrejön, bejelentkezés csak jóváhagyás után lehetséges	Létrejött	Igen
R003	Regisztráció már létező e-mail tiltása	Már beregisztrált e-mail címmel nem lehet újabb regisztrációt véghez vinni	Hibaüzenet jelenik meg	Megjelent	Igen
R004	Regisztráció már létező felhasználónév tiltása	Már beregisztrált felhasználónévvel nem lehet újabb regisztrációt véghez vinni	Hibaüzenet jelenik meg	Megjelent	Igen

Bejelentkezés					
B001	Bejelentkezés helyes adatokkal	Megadom az e-mail címet és a jelszót	Belépés a főoldalra, felül látszik a felhasználónév	Működik	Igen
B002	Bejelentkezés moderátorral	Jóváhagyott moderátor szerepkörű felhasználó e-mail címmel és jelszóval bejelentkezik	Belépés a főoldalra, felül látszik a felhasználónév, valamint a Moderálás menü	Működik	Igen
B003	Bejelentkezés hibás jelszóval	Rossz jelszó beírása	Hibaüzenet jelenik meg	Működik?	Igen
Kérdés létrehozása					
K001	Kérdés létrehozásánál üres mezők tiltása	Üresen hagyott kérdés cím és/vagy leírás beküldése	Hibaüzenet jelenik meg a mezők alatt	Működik	Igen
K002	Kérdés létrehozása nem lehetséges bejelentkezés nélkül	Vendég felhasználó, aki az oldalon böngészik nem tud kérdést beküldeni	Felugró ablakban kéri a rendszer a bejelentkezést	Működik	Igen
K003	Kérdés létrehozásánál trágár szavak tiltása	Kérdés címében és/vagy leírásában trágár kifejezés megadása	Trágár kifejezést tartalmazó szöveg beküldése után figyelmeztetés jelenik meg, javítani kell	Működik	Igen
K004	Kérdés címkével való létrehozása	Kérdés létrehozásánál címke beírása és beküldése	Egy, vagy több címke megadása és beküldése a kérdés létrehozásakor	Üres címkénél hiba.	Nem
K005	Új kérdés beküldésekor tájékoztató üzenet a beküldésről	Megfelelően kitöltjük az űrlapot és beküldés gombra kattintunk	A beküldés gombra kattintva felugró üzenetben tájékoztatja a felhasználót, hogy sikeres a beküldés és csak moderátori jóváhagyás után fog megjelenni	Működik	Igen

Teszt kód	Leírás	Lépések	Elvárt eredmény	Eredmény	Sikeres?
Válasz létrehozása					
V001	Válasz létrehozásánál üres mezők tiltása	Üresen hagyott válasz beküldése	Hibaüzenet jelenik meg a mező alatt	Működik	Igen
V002	Válasz létrehozása nem lehetséges bejelentkezés nélkül	Vendég felhasználó, aki az oldalon böngészik, nem tud választ beküldeni bejelentkezés nélkül	Felugró ablakban kéri a rendszer a bejelentkezést	Működik	Igen
V003	Válasz létrehozásánál trágár szavak tiltása	Válasz mezőben megadott trágár kifejezés előfordul	Trágár kifejezést tartalmazó szöveg beküldése után figyelmeztetés jelenik meg, javítani kell, addig nem kerül beküldésre	Működik	Igen
V004	Új válasz beküldésekor tájékoztató üzenet a beküldésről	Megfelelően kitöltjük a válasz mezőt és beküldés gombra kattintunk	A beküldés gombra kattintva felugró üzenetben tájékoztatja a felhasználót, hogy sikeres a beküldés és csak moderátori jóváhagyás után fog megjelenni	Működik	Igen
V005	Új kérdés beküldésekor tájékoztató üzenet a beküldésről	Megfelelően kitöltjük az űrlapot és beküldés gombra kattintunk	A beküldés gombra kattintva felugró üzenetben tájékoztatja a felhasználót a moderátori jóváhagyásról	Működik	Igen

Teszt kód	Leírás	Lépések	Elvárt eredmény	Eredmény	Sikeres?
Moderátor oldal					
M001	Moderátor oldalon várakozó kérdés moderálása	Moderátor megtekinthet/jóváhagyhat/Törölhet beküldött kérdést	Mindhárom funkció működik, felugró üzenetben tájékoztatja a moderátort/admint	Csak jóváhagyott kérdést engedett	Nem
M002	Moderátor menü várakozó válasz moderálása	Moderátor megtekinthet/jóváhagyhat/Törölhet beküldött választ	Mindhárom funkció működik, felugró üzenetben tájékoztatja a moderátort/admint	Nem jóváhagyott válasz azonnal megjelent publikus oldalon.	Nem
M003	Moderátor menü várakozó moderátorok jóváhagyása admin számára	Moderátorként regisztrált felhasználók jóvhagyása/elutasítása	Mindkettő funkció működik admin számára	Működik	Igen
M004	Moderátor menü várakozó moderátorok jóváhagyása moderátor számára	Moderátorként regisztrált felhasználók jóváhagyása/elutasítása moderátor számára	Moderátor szerepkörű bejelentkezett felhasználó nem látja, hogy mely moderátori kérelmek várakoznak, így jóváhagyni/törölni sem tud (későbbi fejlesztés miatt látható számukra ez a rész)	Működik	Igen

Teszt kód	Leírás	Lépések	Elvárt eredmény	Eredmény	Sikeres?
Keresés					
L001	Szabadmezőben beírt szó(részlet) szerinti keresés	Keresés mezőben beírt szó(részlet) beírása, enter	Minden rendelkezésre álló kérdést listáz, amelyben a beírt szó(részlet) szerepel	Működik	Igen
L002	Témakör szerinti szűrés	Témakörök közül egyik gombra kattintva szűri a kérdések listáját	Minden rendelkezésre álló kérdést listáz, amely az adott témakörben szerepel	Működik	Igen
L003	Címke szerinti szűrés	Népszerű címkék közül egyikre kattintva szűri a kérdések listáját	Minden rendelkezésre álló kérdést listáz, amely az adott címkével ellátott	Működik	Igen
Szavazat					
S001	Szavazat létrehozása nem lehetséges bejelentkezés nélkül	Vendég felhasználó, aki az oldalon böngészik, nem tud kérdésre, válaszra szavazni	Felugró ablakban kéri a rendszer a bejelentkezést a szavazat beküldéséhez	Működik	Igen
S002	Kérdésre szavazás	Bejelentkezett felhasználó szavaz egy kérdésre, amire még nem szavazott	Felugró ablakban tájékoztat a sikeres szavazásról	Működik	Igen
S003	Kérdésre szavazás	Bejelentkezett felhasználó szavaz egy kérdésre, amire már szavazott	Felugró ablakban tájékoztat, hogy erre már szavazott	Működik	Igen
S004	Kérdésre szavazás	Bejelentkezett felhasználó szavaz a saját kérdésére	Felugró ablakban tájékoztat, hogy saját maga által beküldött kérdésre nem szavazhat	Működik	Igen

Teszt kód	Leírás	Lépések	Elvárt eredmény	Eredmény	Sikeres?
S005	Válaszra szavazás	Bejelentkezett felhasználó szavaz egy válaszra, amire még nem szavazott	Felugró ablakban tájékoztat a sikeres szavazásról	Működik	Igen
S006	Válaszra szavazás	Bejelentkezett felhasználó szavaz egy válaszra, amire már szavazott	Felugró ablakban tájékoztat, hogy erre már szavazott	Működik	Igen
S007	Válasz megjelölése elfogadott válasz-nak	Kérdést beküldő kiválaszthat a válaszok közül elfogadott választ a gombra kattintva	Felugró ablakban tájékoztat a sikeres elfogadott válaszról	Route hibára fut.	Nem
S008	Elfogadott válasz visszavonása	Kérdést beküldő visszavonhatja a korábban elfogadott választ a gombra kattintva	Felugró ablakban tájékoztat a sikeres elfogadott válasz visszavonásáról	Nem megfelelő működés	Nem

Hibajegyzék

Teszt kód	Leírás	Állapot	Javítás
M001	A „Megtekintés” gomb nem működött nem jóváhagyott kérdésnél.	A KerdesController.Reszletek() csak jóváhagyott kérdést engedett, emiatt moderátor is NotFound oldalt kapott.	Szerepkör-ellenőrzés került be
M002	Moderátor menü várakozó válasz moderálása	Nem jóváhagyott válasz azonnal megjelent publikus oldalon - minden választ betöltött.	Nem admin felhasználónál szűrés
M001	Kérdés törlésekor exception	Jóváhagyásra váró kérdés törlésnél exception. Kiderült, hogy Kapcsolódó rekordok maradtak: KerdesCimkek, Valaszok, Szavazatok	Teszt során idegen kulcs miatti törlési hiba jelentkezett, kapcsolódó rekordok törlésével javítva.
S007	Elfogadott válasz kijelölése	Route hibára fut.	ValaszController elfogadás logika javítva.
S008	Elfogadott válasz visszavonása	Nem megfelelő működés	ElfogadasVisszavon() ellenőrzések korrigálása.
K004	Kérdés címkével létrehozás	Tiltott címke validáció + címke kapcsolás finomítás történt.	Címke lehet opciionális.

3. Felhasználói dokumentáció

A KertKérdés alkalmazás webböngészőből használható, ezért a végfelhasználó számára nincs szükség fejlesztői környezet telepítésére, amennyiben az alkalmazás szerveroldalon publikált formában fut. A felhasználó a rendszerhez böngészőn keresztül kapcsolódik, ahol a felület dinamikusan jeleníti meg az adatbázisban tárolt kérdéseket, válaszokat és egyéb kapcsolódó információkat.

A rendszer használata során a felhasználó szerepkörétől függően eltérő funkciók válnak elérhetővé. Az alapfelhasználók kérdéseket tehetnek fel, válaszolhatnak és böngészhetik a tartalmakat, míg magasabb jogosultsági szinten további moderálási lehetőségek jelennek meg. Ezáltal a rendszer egyszerre szolgál közösségi tudásmegosztó felületként és ellenőrzött tartalomkezelő környezetként.

A következő alfejezetek részletesen bemutatják az alkalmazás elérésének feltételeit, az indítás módját, valamint a fő felhasználói felületek működését képernyőképekkel és magyarázatokkal együtt.

Fejlesztői környezetben történő futtatás

A fejlesztés és tesztelés során az alkalmazás Visual Studio környezetből futtatható.

Ehhez szükséges:

- Visual Studio 2022
- .NET 8 SDK
- SQL Server
- a projekt teljes forráskódja

Ebben az esetben a fejlesztő közvetlenül a projektet nyitja meg, build-eli, majd helyi környezetben futtatja.

A rendszer ilyenkor általában:

- localhost címen érhető el
- automatikusan csatlakozik a helyi adatbázishoz

Ez a módszer fejlesztéshez és hibakereséshez ideális.

Első indítás menete

1. adatbázis létrehozása (SQL Serveren kertkerdes login létrehozása, ezután az adatbázis létrehozásához script futtatása)

2. connection string beállítása appsettings.json fájlban

3. projekt build

4. futtatás

Belépés menete

A felhasználó regisztráció után beléphet. Belépéskor:

- session jön létre,
- jogosultság ellenőrződik.

Végfelhasználói használat – publikált alkalmazásként

A publish folyamat során a Visual Studio a teljes projektből elkészít egy futtatható telepített verziót.

Ez tartalmazza:

- lefordított DLL állományokat
- konfigurációs fájlokat
- statikus webes tartalmakat
- futtatáshoz szükséges szerkezetet

Publish eredménye

A publish után létrejön egy külön mappa, amiben megtalálható például:

- KertKerdes.dll
- web.config
- wwwroot
- appsettings.json

Ez már telepíthető szerverre.

IIS használata

A publikált ASP.NET Core alkalmazás egyik leggyakoribb futtatási környezete az Internet Information Services. Az IIS a Windows operációs rendszer beépített webszervere.

Feladata:

- HTTP kérések fogadása
- az alkalmazás kiszolgálása böngésző felé
- kapcsolat fenntartása a háttérfolyamattal

Hogyan működik IIS alatt?

Az IIS közvetlenül nem a C# kódot futtatja, hanem az ASP.NET Core Runtime segítségével indítja el az alkalmazást.

Ehhez szükséges:

- ASP.NET Core Hosting Bundle telepítése
- publikált állományok bemásolása
- webhely létrehozása IIS-ben

Ezáltal a felhasználó számára már csak böngésző kell. Ha az alkalmazás már publikálva van szerveren, akkor a felhasználó egyszerűen URL címen éri el. A szerver:

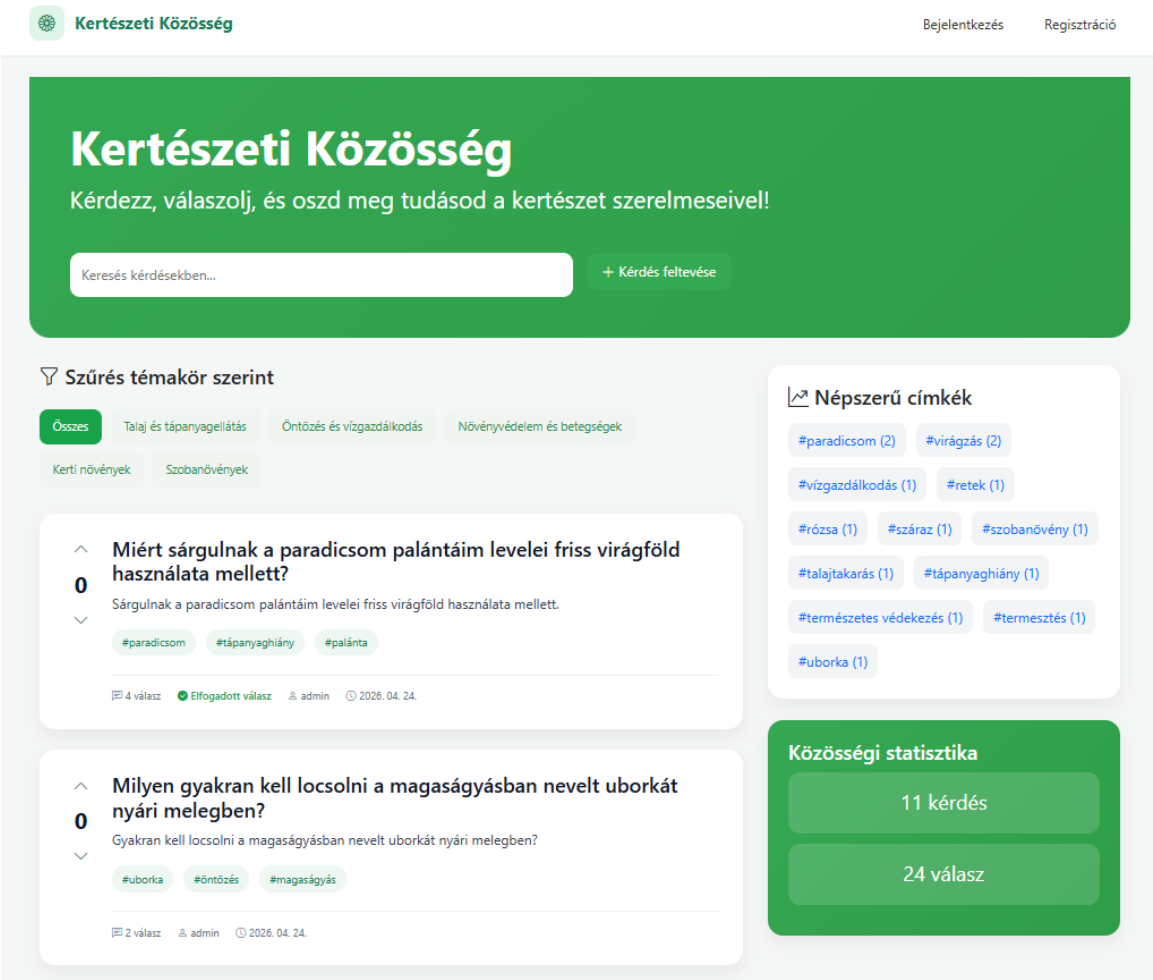
- fogadja a felhasználó kérését
- továbbítja az ASP.NET Core alkalmazásnak
- az alkalmazás lekérdezi az adatbázist
- elkészíti a HTML oldalt
- visszaküldi böngészőbe

3.3 Felületek bemutatása

A KertKérdés webalkalmazás felhasználói felülete úgy került kialakításra, hogy a rendszer funkciói logikusan elkülönüljenek, ugyanakkor könnyen kezelhetők maradjanak olyan felhasználók számára is, akik nem rendelkeznek informatikai háttérrel. A navigáció célja az volt, hogy a kérdésfeltevés, válaszadás, keresés és moderáció egyértelmű útvonalakon keresztül történjen.

A felhasználói felület Razor alapú nézetekből épül fel, amelyek dinamikusan jelenítik meg az adatbázisból érkező adatokat. Minden oldal a controller-ekből kapott adatmodellek alapján töltődik fel, ezért a felület mindig az aktuális adatbázisállapotot tükrözi.

főoldal



A főoldal a rendszer központi felülete, ahol a felhasználó elsőként találkozik az alkalmazás tartalmával. Itt jelennek meg a jóváhagyott kérdések listázott formában.

A kérdések megjelenítése során minden rekordhoz kapcsolódóan látható:

- a kérdés címe,
- rövid leírás vagy kivonat,
- a létrehozó felhasználó neve,
- a feltöltés időpontja,
- a kapcsolódó témakör.

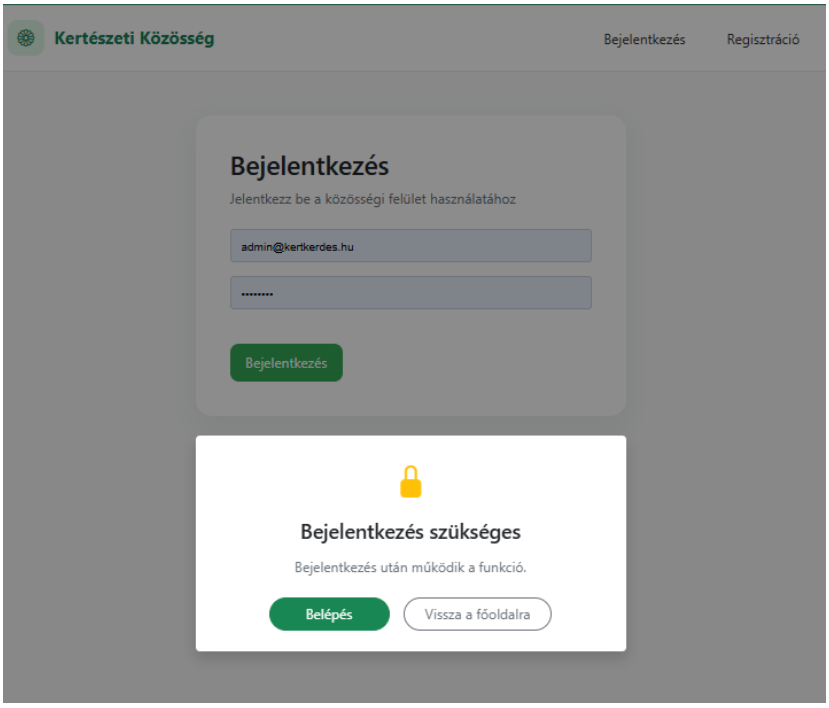
A lista célja, hogy a felhasználó gyors áttekintést kapjon a legfrissebb vagy legfontosabb tartalmakról.

A főoldalon ezen kívül statisztikai adatok is megjelennek:

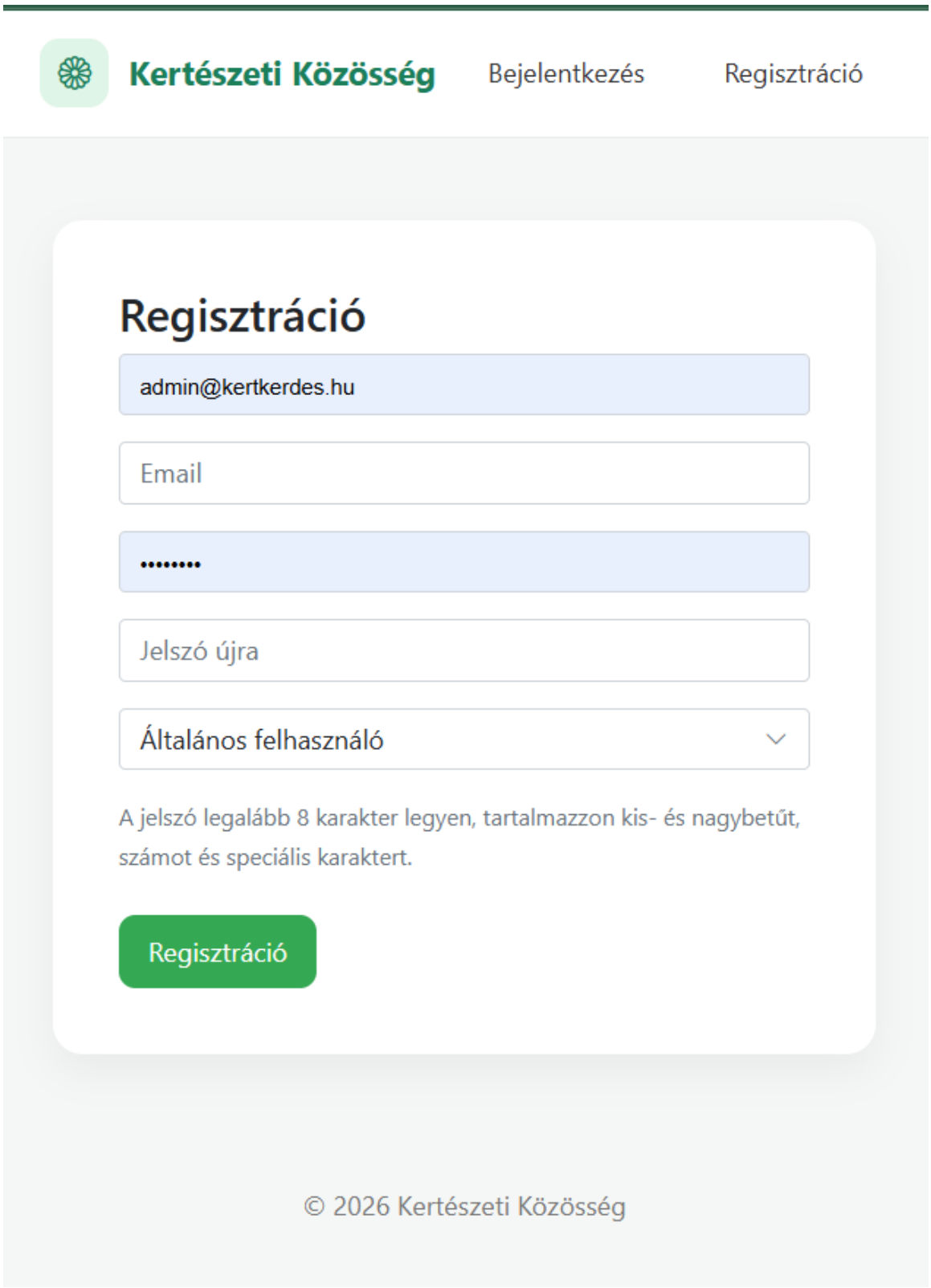
- hány kérdés található a rendszerben,
- hány válasz került jóváhagyásra.

Ez a funkció azért hasznos, mert a felhasználó azonnal érzékeli, hogy aktív közösségi rendszerben van. A háttérben a **KerdesController** végzi a kérdések lekérdezését, és csak azokat a rekordokat jeleníti meg, amelyek jóváhagyott állapotúak.

Fontos megjegyeznünk, hogy a vendég felhasználó, aki bejelentkezés nélkül böngészik az oldalon szavazat leadásakor, illetve kérdés feltevése gombra kattintva értesítést kap, hogy ezek a funkciók csak regisztrált felhasználók számára elérhetőek. Ezáltal rávehetőek, hogy a rendszer regisztrációt követően vegyék igénybe.



regisztráció



A regisztrációs oldal célja új felhasználók létrehozása a rendszerben.

A felhasználónak itt kötelezően meg kell adnia:

- felhasználónevét,
- email címét,
- jelszavát.

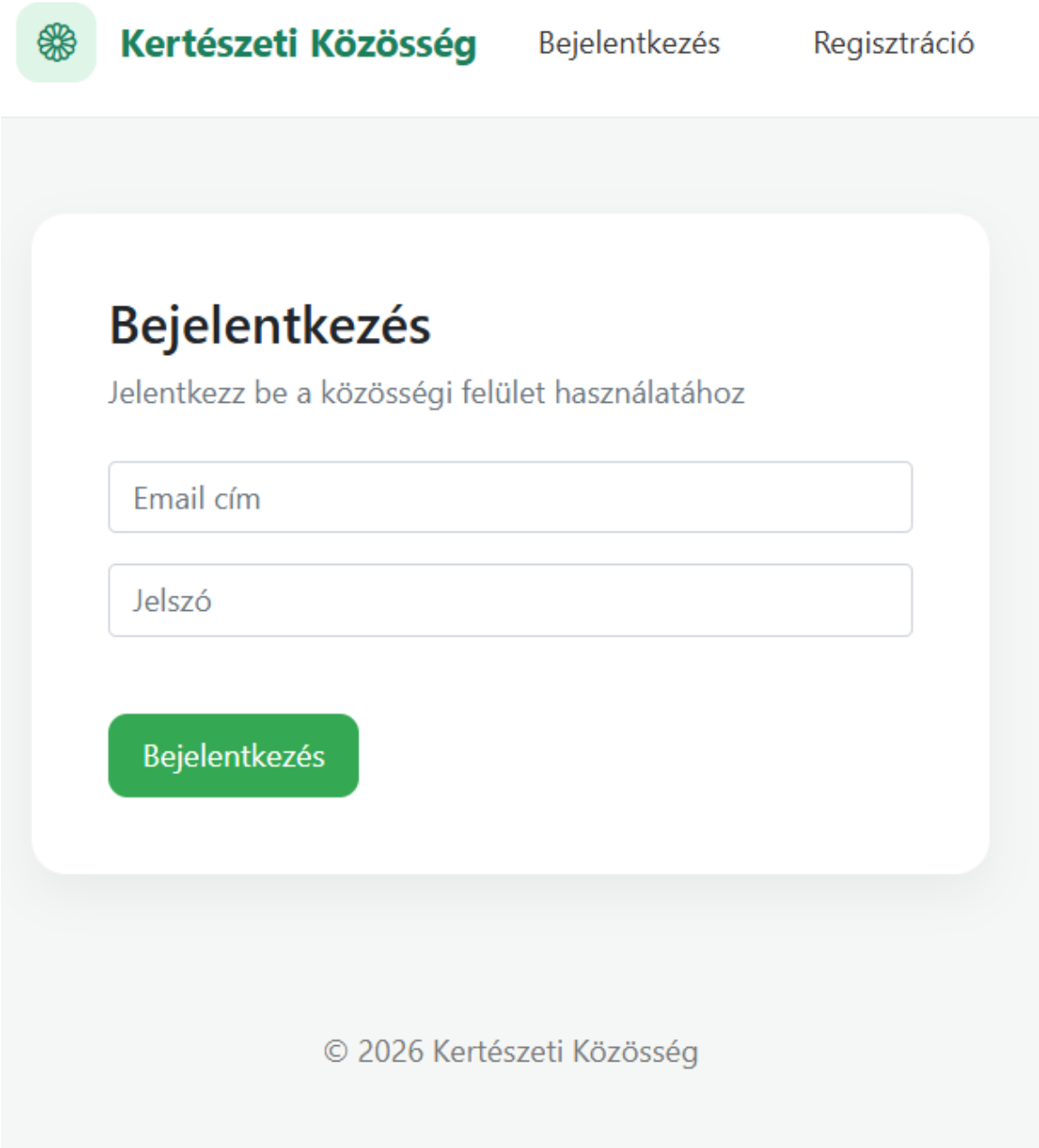
A rendszer a regisztráció pillanatában több ellenőrzést végez:

- email ellenőrzése: megvizsgálja, hogy szerepel-e már az adatbázisban.
- felhasználónév ellenőrzése: biztosítja az egyediséget.
- jelszó feldolgozása: a jelszó nem közvetlenül kerül eltárolásra, hanem SHA256 hash készül belőle.

Ez azért fontos, mert az adatbiztonság egyik alapelve, hogy a felhasználói jelszavak ne legyenek visszafejthető formában elérhetők.

A regisztráció után a felhasználó rekordként bekerül a **Felhasznalok** táblába.

bejelentkezés



A belépési felületen a felhasználó email címével és jelszavával tud belépni a rendszerbe.

A belépés során a rendszer:

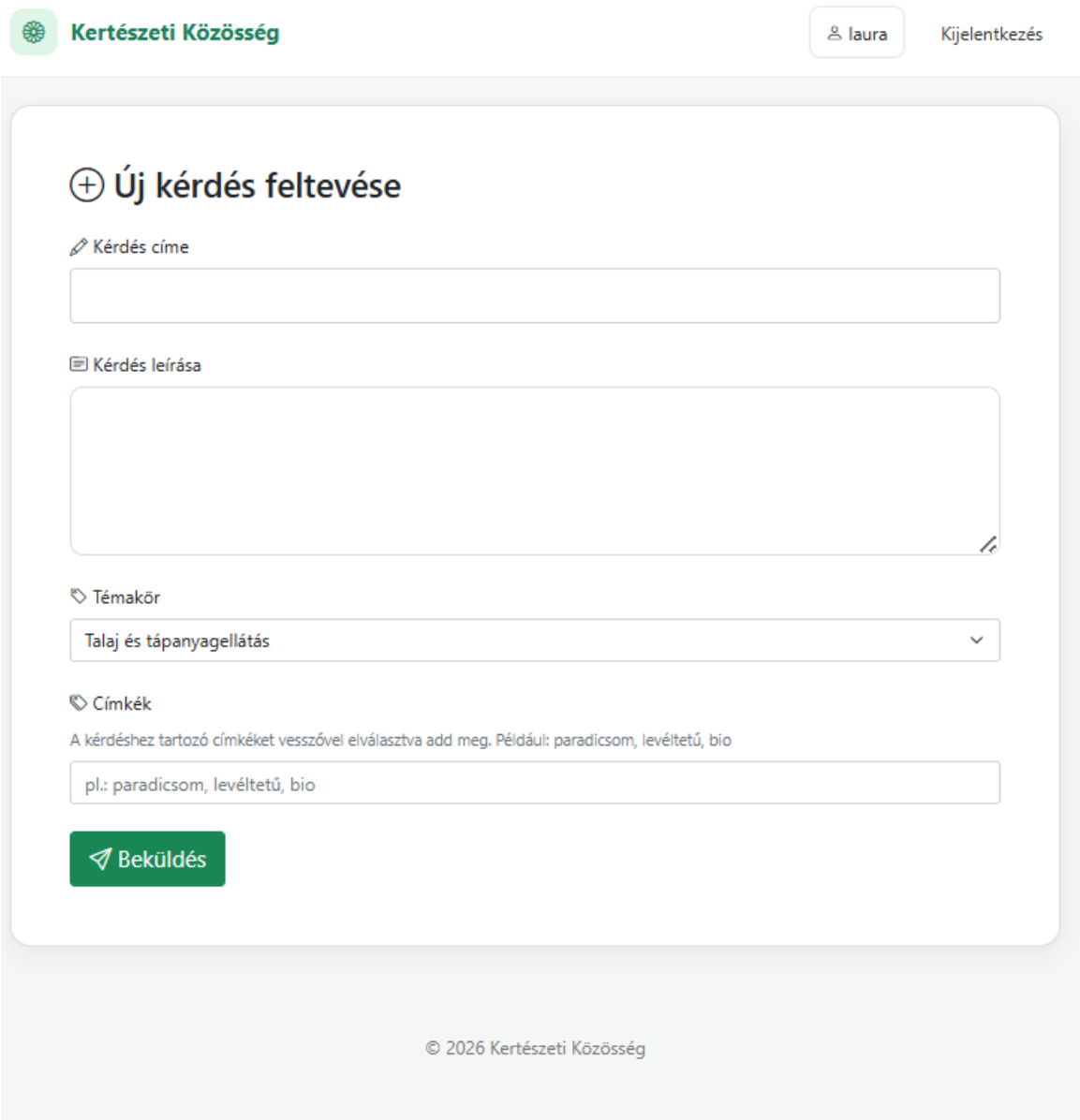
- megkeresi a felhasználót email alapján,
- hash-eli a megadott jelszót,
- összehasonlítja az adatbázisban tárolt hash értékkel.

Sikeres hitelesítés után session jön létre. Ez a session tárolja:

- a felhasználó azonosítóját,
- felhasználónevét,
- jogosultsági szintjét.

Ennek köszönhetően a rendszer később felismeri, hogy ki van belépve.

kérdés létrehozás



Ez a felület a rendszer egyik legfontosabb része. A felhasználó itt új kérdést tud feltenni.

A mezők:

- kérdés címe,
- részletes leírás,
- témakör kiválasztása,
- címkék megadása.

A címkék célja, hogy a későbbi keresést segítsék.

Például:

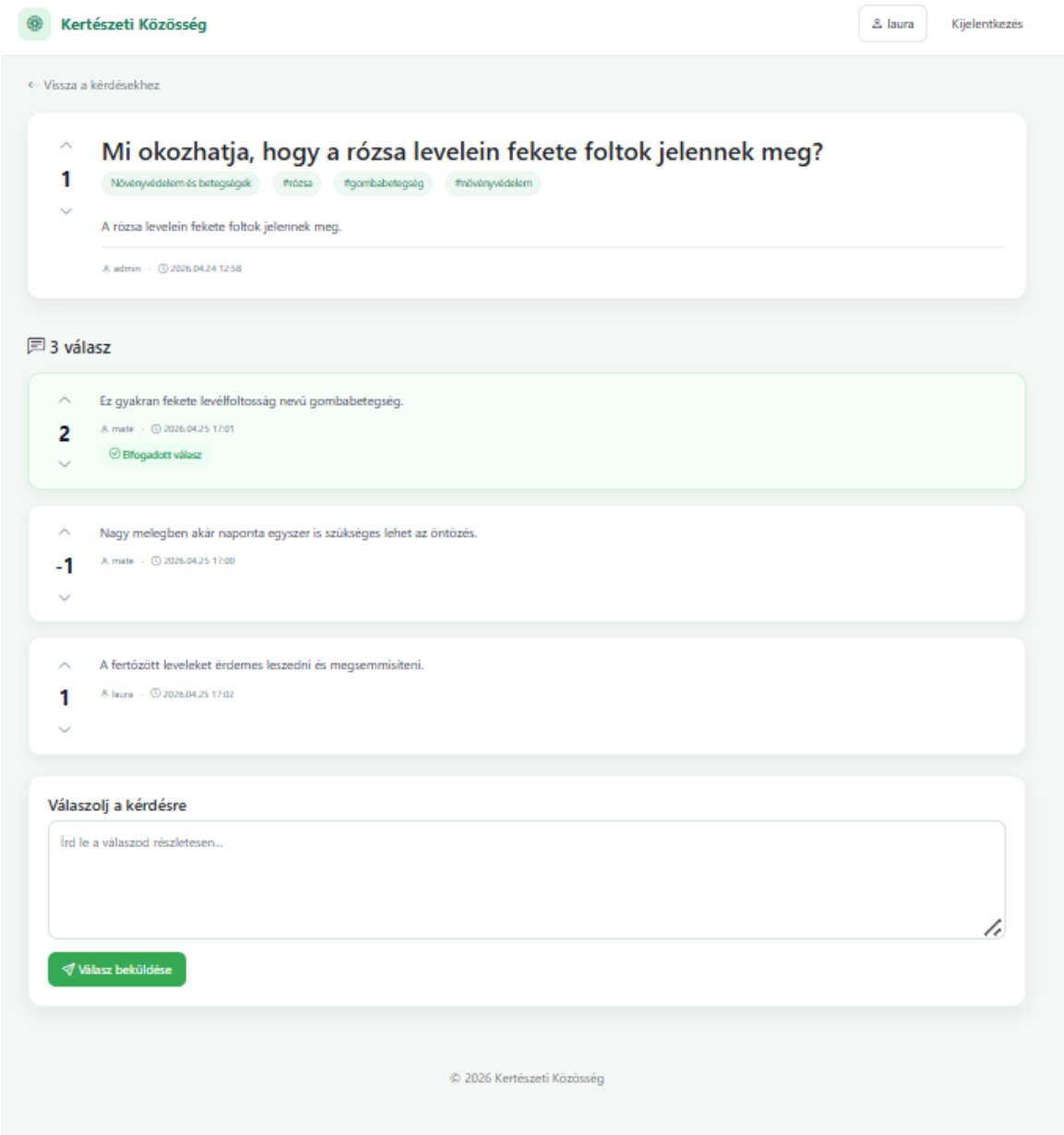
- paradicsom
- metszés
- öntözés

A rendszer a beküldéskor ellenőrzi a kötelező mezők kitöltését, a tiltott szavak szűrését, a címkék normalizálását.

A kérdés nem jelenik meg azonnal publikus formában, hanem moderációra váró állapotban kerül mentésre.

Ez azért fontos, mert így a tartalom ellenőrzött módon kerül a nyilvános felületre.

kérdés részletek



Ez az oldal jeleníti meg egy konkrét kérdés teljes tartalmát. Itt látható:

- a teljes kérdésszöveg,
- a szerző neve,
- a címkék,
- a válaszok listája.

A válaszok időrendben jelennek meg. Amennyiben létezik elfogadott válasz, az kiemelve közvetlenül a kérdés alatt jelenik meg.

A válaszadás egyszerű szöveges mező segítségével történik. Beküldés után a válasz:

- bekerül adatbázisba,
- jóváhagyásra vár.

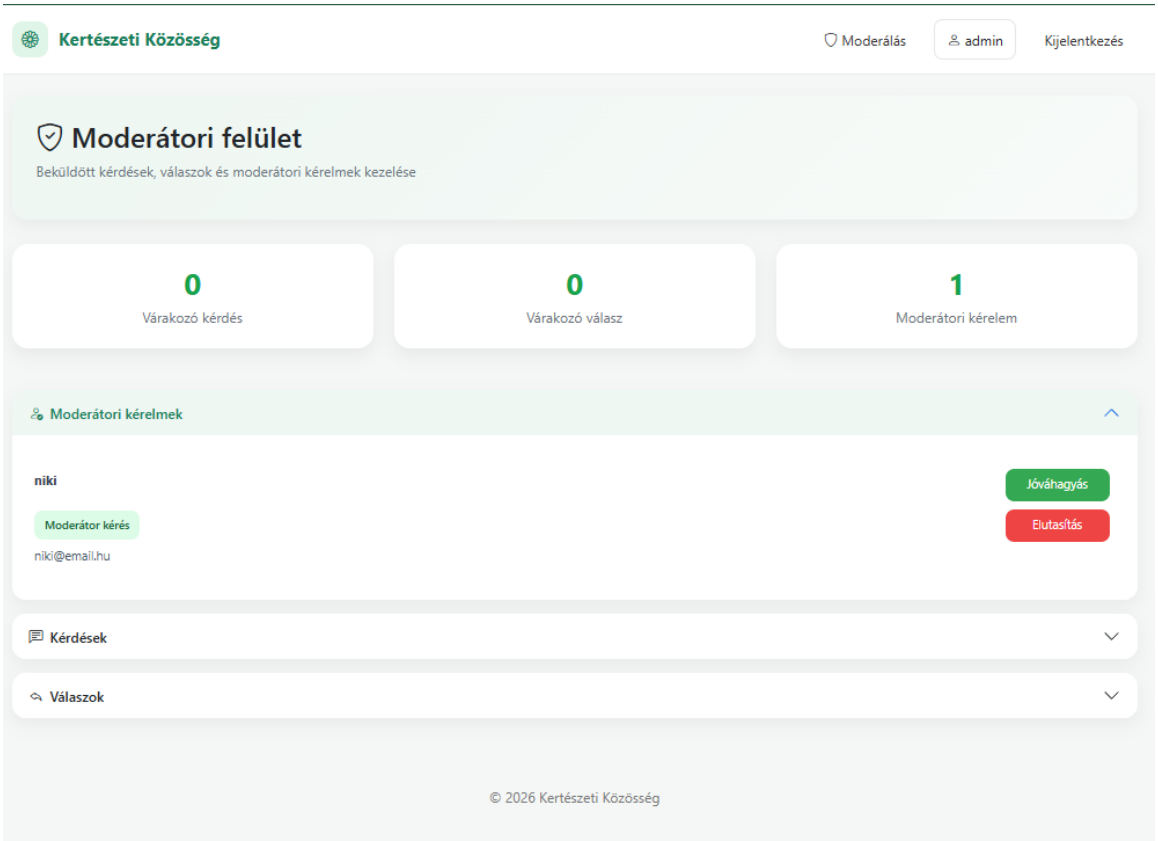
Ez ugyanazt a moderációs logikát követi, mint a kérdések.

A rendszer lehetőséget ad pozitív vagy negatív szavazatra. A szavazás célja, hogy a hasznos tartalom kiemelkedjen.

A háttérben minden szavazat külön rekordként kerül a Szavazatok táblába. A rendszer ellenőrzi, hogy egy felhasználó ugyanarra az elemre csak egyszer szavazhasson. A rendszert úgy terveztük meg, hogy a kérdést beküldő nem tud szavazatot leadni saját maga beküldött kérdésére.

Ez a felület mutatja meg legjobban a rendszer közösségi működését.

moderálás felület



A rendszer külön moderálási felületet biztosít a közösségi tartalmak ellenőrzésére, amelynek célja, hogy a felhasználók által beküldött kérdések és válaszok csak ellenőrzés után jelenjenek meg nyilvánosan. Ez a felület nem általános adminisztrációs oldal, hanem kifejezetten a tartalom jóváhagyására és szűrésére szolgáló munkafelület.

A moderálási felülethez azok a felhasználók férnek hozzá, akik rendelkeznek megfelelő jogosultsággal, vagyis:

- moderátor jogosultságú felhasználók,
- admin jogosultságú felhasználók.

Ez azt jelenti, hogy a rendszer nem kizárólag egyetlen adminisztrátorra épül, hanem lehetővé teszi több tartalomellenőrző felhasználó egyidejű működését is.

A felület elsődleges feladata a még nem jóváhagyott tartalmak listázása. Itt külön jelennek meg a jóváhagyásra váró kérdések, valamint válaszok.

Minden rekordhoz tartozik egy ellenőrzési lehetőség, amely segítségével a jogosult felhasználó eldöntheti, hogy a tartalom megfelel-e a rendszer szabályainak. A moderátor szerepkörrel rendelkező felhasználó jogosult:

- kérdések jóváhagyására
- kérdések elutasítására vagy törlésére
- válaszok jóváhagyására
- válaszok törlésére

Ez a funkció azért fontos, mert a közösségi rendszerben megjelenő tartalom minőségét így folyamatosan ellenőrizni lehet.

Jóváhagyás után a rendszer a kiválasztott rekordnál módosítja a jóváhagyási állapotot. Technikailag ez a háttérben a megfelelő rekord Jovahagyva mezőjének módosításával történik. A jóváhagyott tartalom ezután automatikusan megjelenik a nyilvános kérdéslistában.

Ha a moderátor úgy dönt, hogy a tartalom nem megfelelő, a rekord törlésre kerül. A törlés során a rendszer figyelembe veszi a kapcsolódó adatokat is:

- válaszokat,
- szavazatokat,
- címkekapcsolatokat.

Ez azért fontos, hogy az adatbázis konzisztens maradjon.

A projekt jogosultsági rendszere több szintet különböztet meg. A **moderátor** elsődleges feladata a tartalmak ellenőrzése. Jogosultsága az általános szerepkörű felhasználóhoz képest, hogy kérdéseket és válaszokat minősít és az alapján hagyja jóvá, illetve törli publikálás előtt.

Az **admin** magasabb szintű jogosultsággal rendelkezik. Nemcsak moderálni tud, hanem felhasználói jogosultságokat is kezelhet. A rendszerben különösen fontos szabály, hogy moderátori jogosultság nem aktiválódik automatikusan. Ha egy felhasználó moderátori szerepkörre jelentkezik, annak aktiválását kizárólag admin jogosultságú felhasználó végezheti el. Ez azért fontos, mert így a rendszer biztosítja, hogy moderátori jogosultság csak ellenőrzött módon kerülhessen kiosztásra.

Ez a jogosultsági modell jól elkülöníti a tartalomellenőrzést és jogosultságkezelést. A moderátor a napi tartalomkezelést végzi, míg az admin a rendszer feletti magasabb szintű felügyeletet biztosítja.

4. Összefoglalás

A projekt során sikerült létrehozni egy működő, adatbázis-alapú, több funkcióból álló webalkalmazást, amely valós felhasználói folyamatokat modellez.

A rendszer képes:

- felhasználók kezelésére,
- kérdések és válaszok tárolására,
- moderációra,
- címkézésre,
- szavazásra.

A fejlesztés során különösen fontos eredmény volt, hogy a projekt modern MVC architektúrára épül, amely jól elkülöníti az adatkezelést, az üzleti logikát és a megjelenítést.

Továbbfejlesztési lehetőségek

A jövőben a rendszer tovább bővíthető:

- email alapú hitelesítéssel
- profiloldallal
- képfeltöltéssel
- fejlettebb kereséssel
- admin statisztikai felülettel

A fejlesztési folyamat során a feladatok megosztása jól működött, mivel az alkalmazás különálló rétegei lehetővé tették, hogy az egyes fejlesztési feladatok párhuzamosan történjenek. A közös munka során folyamatos egyeztetés biztosította, hogy a rendszer egységes szerkezetben készüljön el.